Modulo 2

Introduzione a Python e Concetti fondamentali

Introduzione a Python e concetti fondamentali

- Introduzione al linguaggio Python e alle sue caratteristiche principali.
- Apprendimento dei concetti fondamentali di programmazione, come variabili, operatori, stringhe, input e output, condizionali, cicli, eccezioni e funzioni.
- Introduzione alla struttura del codice Python, come moduli, pacchetti e commenti.

Introduzione a Python e concetti fondamentali

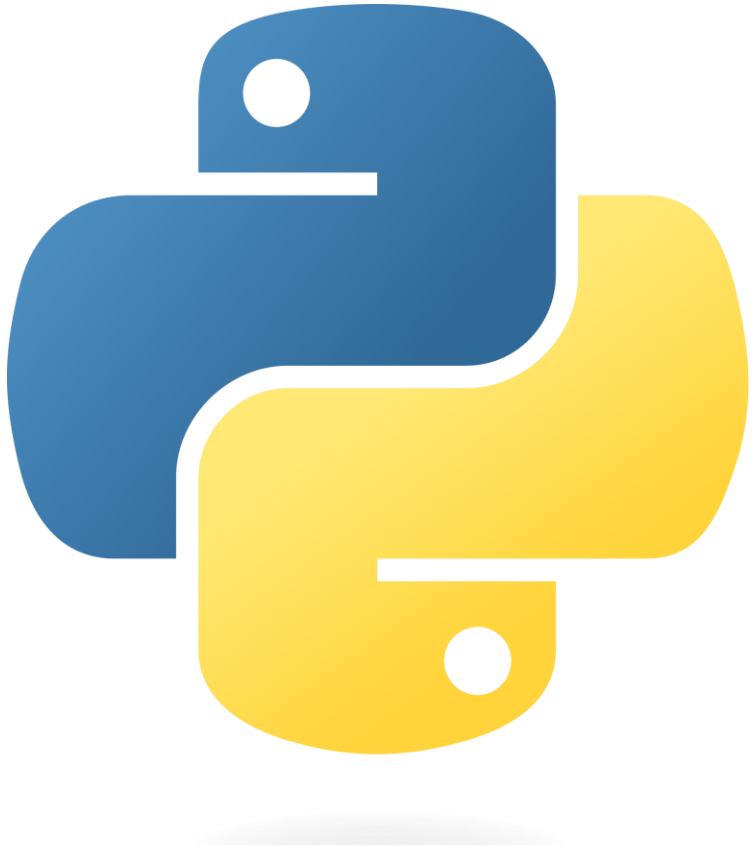
- Storia e filosofia di Python
- Installazione di Python e configurazione dell'ambiente
- di sviluppo
- Hello World: il tuo primo programma in Python
- Caratteristiche principali del linguaggio
- Python 2 vs Python 3: principali differenze
- La Python Software Foundation e la community
- Interpretato vs compilato: come funziona Python
- Uso dell'interprete interattivo (shell)
- Introduzione a pip e gestione dei pacchetti
- Buone pratiche e stile di codice (PEP 8)
- Strumenti di sviluppo e IDEs popolari
- Variabili e tipi di dati
- Operatori: aritmetici, confronto, logici
- Stringhe: proprietà e metodi associati
- Input ed output: leggere e stampare dati
- Operazioni condizionali (if, elif, else)
- Cicli (for, while) e iterazione
- Gestione delle eccezioni (try, except, finally)
- Funzioni: definizione, parametri, return
- Scope delle variabili e namespaces
- Moduli e pacchetti: organizzazione del codice
- Introduzione ai commenti e alla documentazione

Modulo 2

Storia e filosofia di Python

PYTHON: Linguaggio di programmazione ad alto livello orientato ad oggetti.

STORIA: Python è stato creato da Guido van Rossum nel 1989 al CWI (Centrum Wiskunde & Informatica) nei Paesi Bassi con l'obiettivo di rendere la programmazione facile. La prima versione pubblica è stata rilasciata nel 1991 come Python 0.9.0. Il nome "Python" è stato ispirato dai Monty Python, un gruppo comico britannico.



THE ZEN OF PYTHON: Il PEP 20 elenca principi guida chiave per la filosofia di Python, tra cui la leggibilità del codice, la semplicità, l'esplicità, la chiarezza e la promozione della diversità nello stile di programmazione. In sintesi, Python enfatizza la scrittura di codice chiaro e leggibile, consentendo diversi modi di programmazione.

Modulo 2

Installazione di Python e configurazione dell'ambiente di sviluppo

L'installazione di Python e la configurazione dell'ambiente di sviluppo possono variare leggermente a seconda del sistema operativo che stai utilizzando. Ecco come installarlo nelle piattaforme più comuni: Windows, macOS e Linux.

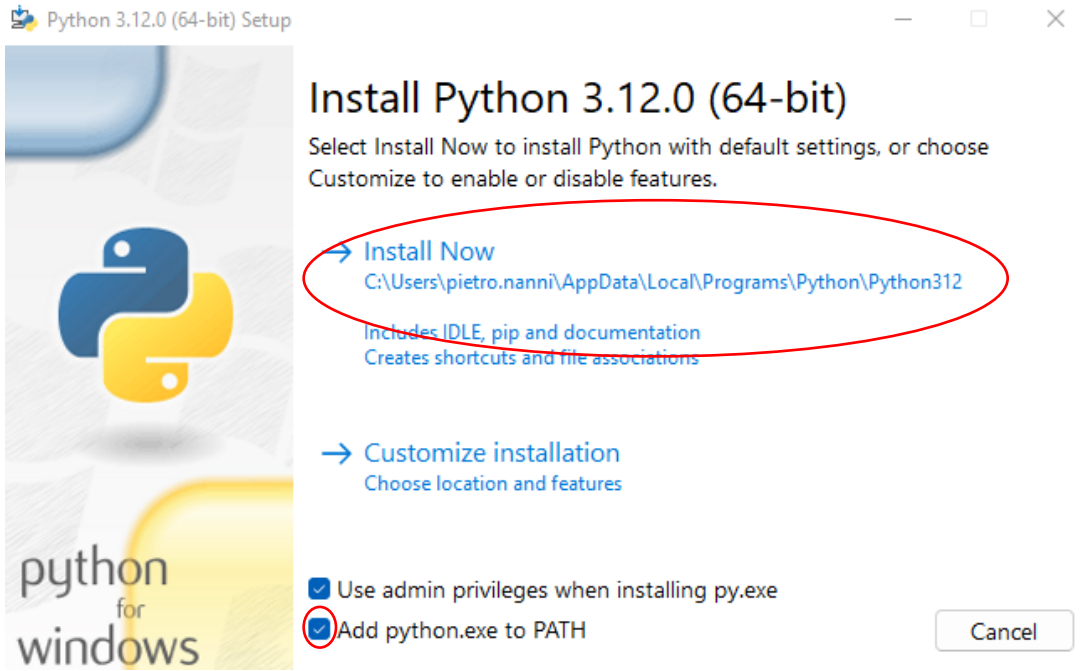
Modulo 2

Windows

1. Download di Python:

Vai al sito ufficiale di Python, python.org, e scarica l'ultima versione di Python per Windows.

Esegui l'installer scaricato. Assicurati di selezionare l'opzione 'Add Python 3.x to PATH' prima di cliccare su 'Install Now'.



Modulo 2

Windows

2. Verifica dell'installazione:

Apri il Command Prompt (cmd) e digita ‘python –version’ per verificare che l'installazione sia stata eseguita correttamente. Dovresti vedere la versione di Python che hai installato.



Modulo 2

Windows

3. Configurazione dell'ambiente di sviluppo (IDE):

Installa un IDE o un editor di testo che supporti Python, come Visual Studio Code, PyCharm, Atom, o Sublime Text.

Configura l'IDE scegliendo l'interprete Python che hai installato.

Modulo 2

Windows

4. (Opzionale) Creazione di un ambiente virtuale:

Nella linea di comando, naviga nella directory del tuo progetto.

Digita 'python -m venv nome_ambiente' per creare un nuovo ambiente virtuale.

Attiva l'ambiente virtuale con 'nome_ambiente\Scripts\activate.bat' su cmd o 'nome_ambiente\Scripts\Activate.ps1 su PowerShell'.

```
C:\Users\admin\Desktop\progetto python>python -m venv
Prova_ambiente_virtuale

C:\Users\admin\Desktop\progetto python>
```

```
C:\Users\admin\Desktop\progetto python>python -m venv Prova_ambiente_virtuale

C:\Users\admin\Desktop\progetto python>
Prova_ambiente_virtuale\Scripts\activate.bat

(Prova_ambiente_virtuale) C:\Users\admin\Desktop\progetto python>
```

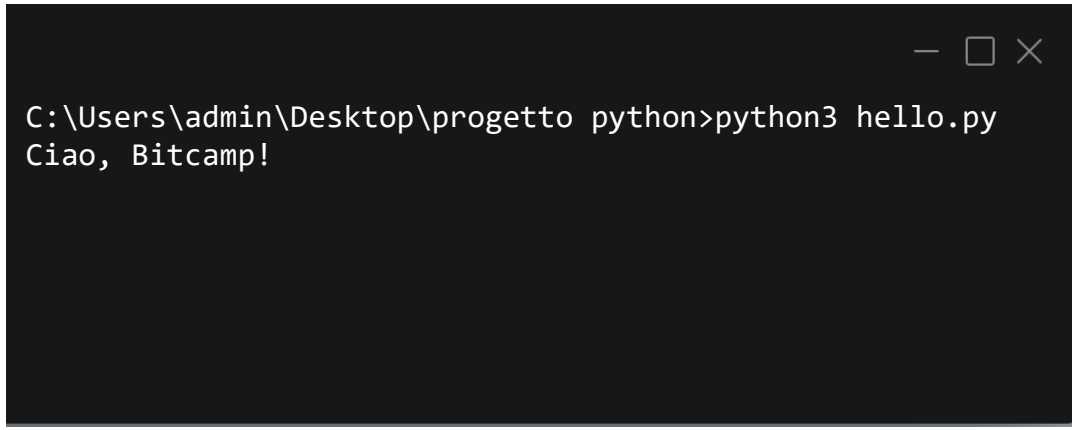
Modulo 2

Ciao Bitcamp: il tuo primo programma in Python

Impara le basi della programmazione con Python.

Apri il tuo editor di testo e preparati a scrivere il tuo codice:

```
print("Ciao, Bitcamp!")
```



Istruzioni passo-passo:

Salva il file con il nome hello.py.
Apri il terminale.
Naviga alla cartella dove hai salvato il file.
Digita `python3 hello.py` e premi Enter.

Modulo 2

Caratteristiche principali del linguaggio

Semplicità e Leggibilità:

- Python favorisce una sintassi pulita e leggibile.
- Le regole di indentazione forzano una scrittura del codice ordinata.
- Questo rende il codice Python più facile da scrivere, leggere e mantenere.

Open Source:

- Python è sviluppato sotto una licenza open source OSI-approved, rendendolo liberamente utilizzabile e distribuibile, anche per uso commerciale.
- La comunità globale contribuisce al suo sviluppo e all'ampio ecosistema di moduli e framework.

Ad Alto Livello:

- Python gestisce automaticamente compiti di basso livello come la gestione della memoria.
- Permette agli sviluppatori di concentrarsi sulla logica del programma anziché sui dettagli dell'architettura della macchina.

Modulo 2

Caratteristiche principali del linguaggio

Multiparadigma:

- Python supporta diversi stili di programmazione, permettendo agli sviluppatori di scegliere il più adatto al problema che stanno affrontando.
- Ciò include paradigmi come la programmazione orientata agli oggetti e funzionale.

Tipizzazione Dinamica e Gestione della Memoria:

- In Python non è necessario dichiarare il tipo di una variabile; questo viene determinato al momento dell'esecuzione.
- Python ha un garbage collector incorporato che si occupa di deallocare la memoria non utilizzata, riducendo il rischio di memory leaks.

Portabilità e Integrazione:

- Python può girare su diverse piattaforme, rendendolo una scelta ideale per applicazioni cross-platform.
- Offre inoltre strumenti di interfacciamento con altri linguaggi di programmazione.

Modulo 2

Caratteristiche principali del linguaggio

Ampie Librerie Standard:

- Python viene fornito con una vasta libreria standard, che aiuta a eseguire molte attività senza la necessità di installare pacchetti aggiuntivi.
- Queste libreria forniscono moduli per interfacciarsi con il sistema operativo, eseguire operazioni sui file, gestire le reti e molto altro.

Utilizzo in Diversi Campi:

- Python trova applicazione in numerosi settori: dall'automazione di compiti semplici al calcolo scientifico, dallo sviluppo web all'intelligenza artificiale.
- La sua versatilità è una delle ragioni principali della sua popolarità.

Facilità di Apprendimento e Utilizzo:

- Grazie alla sua sintassi semplice e chiara, Python è spesso consigliato come primo linguaggio di programmazione.
- Esistono innumerevoli risorse online, come tutorial, corsi e documentazione, che rendono l'apprendimento di Python accessibile a tutti.

Modulo 2

Python 2 vs Python 3: principali differenze

Python 2		Python 3
`print "Hello, world"`	**Stampa di testo**	`print ("Hello, world")`
`3 / 2 = 1`	**Divisione Intera**	`3 / 2 = 1.5 e 3 // 2 = 1`
Stringhe come ASCII di default con ‘u’ per Unicode es.(‘`u"Hello"`’)	**Unicode**	Stringhe come Unicode di default es.(`"Hello"`)
`range()` crea una lista, `xrange()` crea un iteratore	**Funzione range()**	`range()` crea un iteratore
Meno rigida, ad esempio con le eccezioni `except Exception, e:`	**Sintassi**	Più rigida e chiara, ad esempio con le eccezioni `except Exception as e:`
Libreria standard disorganizzata	**Libreria Standard**	Libreria standard riorganizzata e migliorata
Supporto terminato nel 2020	**Fine del Supporto**	Attivamente sviluppato e supportato

Modulo 2

La Python Software Foundation e la community

Python Software Foundation (PSF):

- Organizzazione non profit fondata nel 2001.
- Obiettivo: Promuovere, proteggere e far avanzare il linguaggio di programmazione Python.
- Responsabile della gestione del marchio Python, supporto allo sviluppo e organizzazione di eventi come PyCon.

Ruolo della PSF:

- Sostiene finanziariamente progetti Python e la community.
- Gestisce licenze e legalità relative a Python.
- Promuove la diversità e l'inclusione nella community Python.

Modulo 2

La Python Software Foundation e la community

Forza della Community:

- Globale e diversificata, con milioni di sviluppatori.
- Contribuisce attivamente allo sviluppo di Python attraverso feedback, codice e documentazione.

Contributi della Community:

- Sviluppo di pacchetti e librerie open-source.
- Partecipazione in mailing list, forum online e gruppi di discussione.
- Organizzazione di incontri locali, hackathon e conferenze

Come Partecipare:

- Contribuire a progetti open source su piattaforme come GitHub.
- Unirsi a gruppi locali o regionali di Python.
- Partecipare a conferenze e eventi Python, come PyCon.

Impatto della PSF e della Community:

- Rafforzamento continuo dell'ecosistema Python.
- Supporto alla crescita professionale degli sviluppatori.
- Promozione dell'innovazione e della collaborazione nel campo dell'informatica.

Modulo 2

Interpretato vs compilato: come funziona Python

Caratteristiche di Python come Linguaggio Interpretato:

Esecuzione Diretta:

- Python esegue il codice direttamente, riga per riga, senza la necessità di un passaggio di compilazione preventivo.

Interprete Python:

- Un programma chiamato 'interprete' esegue il codice Python. Legge e esegue il codice sorgente.

Vantaggi:

- Sviluppo più rapido (ciclo edita-testa più veloce).
- Maggiore flessibilità e dinamicità nel codice.

Come Funziona:

- Gli script Python (.py) vengono eseguiti dall'interprete Python.
- L'interprete Python traduce il codice sorgente in un formato intermedio chiamato bytecode, che viene poi eseguito.
- La traduzione avviene in tempo reale, al momento dell'esecuzione.

Modulo 2

Interpretato vs compilato: come funziona Python

Linguaggi Compilati: Esempi e Caratteristiche:

Esempi: C, C++, Java (compilazione in bytecode).

Processo di Compilazione:

- Il codice sorgente viene trasformato in codice macchina (o bytecode) prima dell'esecuzione.
- Questo passaggio è svolto da un programma chiamato 'compilatore'.

Vantaggi dei Linguaggi Compilati:

- Solitamente più veloci all'esecuzione perché il codice è già compilato.
- Maggiore ottimizzazione possibile durante la fase di compilazione.

Python: Miglior Utilizzo e Flessibilità:

- Ideale per scripting, prototipazione rapida, data science e web development.
- La natura interpretata consente una maggiore flessibilità, facilità di debug e iterazione rapida.

Modulo 2

Uso dell'interprete interattivo (shell)

Cosa è l'Interprete Interattivo?

- Un ambiente dove puoi scrivere ed eseguire codice Python in tempo reale.
- Utile per sperimentare, testare e imparare il linguaggio Python.

Come Accedere all'Interprete Interattivo:

- Digitare `python` o `python3` nel terminale (o prompt dei comandi) e premere invio.
- Dovrebbe apparire un prompt che indica >>>, pronto a ricevere input Python.

```
C:\Users\admin\Desktop\progetto
python>python3
Python 3.12 (tags/v3.11.6:8b6ee5b, Nov 10
2023, 14:57:12) [MSC v.1935 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or
"license" for more information.
>>>
```

Caratteristiche Chiave:

- Esecuzione immediata di comandi Python.
- Ottimo strumento per testare frammenti di codice e funzioni.
- Possibilità di visualizzare direttamente i risultati e gli errori.

Modulo 2

Uso dell'interprete interattivo (shell)

Esempi di Comandi Base

- Operazioni matematiche

```
>>> 9*9
81
```

- Stampa di stringhe

```
>>> print("Ciao, Bitcamp!")
Ciao, Bitcamp
```

- Assegnazione ed uso delle variabili

```
>>> x=10
>>> print(x)
10
```

Modulo 2

Uso dell'interprete interattivo (shell)

Consigli Pratici

- Utilizzare **help()** per ottenere informazioni su funzioni e moduli.

```
>>> help()
help>
```

Digitare **exit()** o premere Ctrl+D (Unix) o Ctrl+Z seguito da Enter (Windows) per uscire dall'interprete

```
>>> exit()
C:\Users\admin\Desktop\progetto python>
```

Modulo 2

Introduzione a pip e gestione dei pacchetti

Cos'è pip?

- pip è il gestore di pacchetti standard per Python, che consente di installare e gestire librerie e dipendenze aggiuntive che non sono incluse nella libreria standard di Python.

Perché è Importante?

- Facilita l'installazione e la gestione di librerie e strumenti esterni.
- Accesso a un vasto ecosistema di pacchetti che estendono la funzionalità di Python.

Installazione e Verifica

- Generalmente, pip è incluso nelle versioni di Python superiori a Python 2.7.9 e Python 3.4.
- Per verificarne la presenza, eseguire 'pip –version' o 'pip3 –version' nel terminale.

```
C:\Users\admin\Desktop\progetto python>pip --version
pip 23.2.1 from C:\Users\admin\AppData\Local\Programs\Python\Python312\Lib\site-
packages\pip (python 3.12)
```

Modulo 2

Introduzione a pip e gestione dei pacchetti

Funzionalità Chiave

- Installazione di pacchetti: ‘pip install nome_pacchetto’
- Elenco dei pacchetti installati: ’pip list’
- Aggiornamento di un pacchetto: ‘pip install --upgrade nome_pacchetto’
- Rimozione di un pacchetto: ‘pip uninstall nome_pacchetto’

Modulo 2

Introduzione a pip e gestione dei pacchetti

Uso Pratico di pip

- Esempio di installazione: 'pip install numpy'
- Dopo l'installazione, i pacchetti possono essere importati ed utilizzati nei programmi Python.

```
C:\Users\admin\Desktop\progetto python>pip install numpy
Collecting numpy
  Obtaining dependency information for numpy from https://files.pythonhosted.org/packages/28/75/3b679b41713bb60e2e8f6e2f
87be72c971c9e718b1c17b8f8749240ddca8/numpy-1.26.2-cp312-cp312-win_amd64.whl.metadata
  Downloading numpy-1.26.2-cp312-cp312-win_amd64.whl.metadata (61 kB)
     ---------------------------------------- 61.2/61.2 kB 251.1 kB/s eta 0:00:00
Downloading numpy-1.26.2-cp312-cp312-win_amd64.whl (15.5 MB)
    ---------------------------------------- 15.5/15.5 MB 3.9 MB/s eta 0:00:00
Installing collected packages: numpy
Successfully installed numpy-1.26.2

[notice] A new release of pip is available: 23.2.1 -> 23.3.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Modulo 2

Introduzione a pip e gestione dei pacchetti

Gestione delle Dipendenze

- Mantenimento di requisiti consistenti attraverso il file 'requirements.txt' (ricordiamoci di creare il file requirements.txt e di inserire all'interno le dipendenze)
- Creazione del file: esecuzione di 'pip freeze > requirements.txt'

```
C:\Users\admin\Desktop\progetto python>pip freeze requirements.txt
numpy==1.26.2
```

- Installazione dei pacchetti da requirements.txt: 'pip install -r requirements.txt'

```
C:\Users\admin\Desktop\progetto python> pip install -r requirements.txt
```

Modulo 2

Introduzione a pip e gestione dei pacchetti

Best Practices e Tips:

- Utilizzo di ambienti virtuali per progetti separati, visto che per ogni progetto avrai bisogno di utilizzare dipendenze diverse, è buona norma creare un ambiente virtuale per ogni progetto e installare le dipendenze inerenti a quel progetto.

- Mantenimento aggiornato delle dipendenze per ragioni di sicurezza e compatibilità.

Come creare un ambiente virtuale

Modulo 2

Buone pratiche e stile di codice (PEP 8)

Introduzione a PEP 8:

- PEP 8 sta per Python Enhancement Proposal 8
- È la guida allo stile ufficiale per la formattazione del codice Python.
- Aiuta a mantenere il codice coerente e facilmente leggibile.

Indentazione:

- Utilizzare 4 spazi per livello di indentazione.
- Evitare l'uso di tabulazioni miste e spazi.

Lunghezza Massima della Riga:

- Limitare le righe a 79 caratteri per il codice e 72 per i commenti e i docstring.

Spazi Bianchi nell'Espressione:

- Evitare spazi bianchi extra in situazioni come ‘dict[key]’ o ‘funzione(arg1, arg2)’.

Modulo 2

Buone pratiche e stile di codice (PEP 8)

Importazioni:

- Utilizzare 4 spazi per livello di indentazione.
- Evitare l'uso di tabulazioni miste e spazi.

Convenzioni sui Nomi:

- Nomi di classi in CamelCase.
- Nomi di funzioni e variabili in lowercase_con_underscore.
- Costanti in MAIUSCOLO_CON_UNDERSCORE.

Commenti e Docstring:

- I commenti dovrebbero essere aggiornati con il codice.
- I docstring devono essere presenti per moduli pubblici, funzioni, classi e metodi.

Raccomandazioni Generali:

- Non confrontare booleani con '== True' o '== False'.
- Utilizzare il controllo delle eccezioni piuttosto che testare le condizioni di errore.

Modulo 2

Strumenti di sviluppo e IDEs popolari

Importanza degli Strumenti di Sviluppo:

- Migliorano l'efficienza e la produttività degli sviluppatori.
- Forniscono supporto per debugging, test, gestione del codice.

Strumenti di Debugging e Testing:

- **PDB (Python Debugger)**: Un debugger interattivo per programmi Python.
- **Pytest**: Un framework per semplificare la scrittura di test semplici ma potenti.
- **Unit Testing Framework**: Integrato in Python per testare singole unità di codice sorgente.

Controllo di Versione:

- **Git**: Sistema di controllo di versione distribuito per tenere traccia delle modifiche al codice sorgente durante lo sviluppo software.
- **GitHub/GitLab/Bitbucket**: Piattaforme online che forniscono hosting per il controllo di versione e la collaborazione.

Modulo 2

Strumenti di sviluppo e IDEs popolari

PyCharm

- IDE sviluppato da JetBrains, specifico per Python.
- Offre analisi del codice, un debugger grafico, un tester integrato, integrazione con sistemi di controllo versione



Visual Studio Code (VS Code):

- Editor di codice sorgente leggero ma potente, estensibile tramite plugin.
- Supporto per Python tramite estensioni, come Python extension e IntelliSense.



Spyder:

- IDE orientato alla data science.
- Integra molte librerie di data science come NumPy, SciPy e Matplotlib.



Sublime Text:

- Editor di testo leggero e personalizzabile.
- Supporto per Python tramite pacchetti e plugin.



Modulo 2

Variabili e tipi di dati

Definizione di Variabile:

- Un riferimento a una posizione di memoria utilizzata per memorizzare dei dati.
- In Python, una variabile viene creata nel momento in cui le si assegna un valore

Assegnazione e Tipizzazione Dinamica:

- Sintassi: `nome_variabile = valore`
- Python è un linguaggio a tipizzazione dinamica; il tipo di una variabile può cambiare.
  Esempi:
  - `numero = 5`(qui numero è un intero)
  - `numero = "cinque"` (ora numero è una stringa)

Regole per il Naming delle Variabili:

- Deve iniziare con una lettera o un underscore (_).
- Può includere lettere, numeri e underscores (es. `my_variable1`).
- Python è case-sensitive: `Variabile, variabile, e VARIABILE` sono considerate diverse.

Buone Pratiche:

- Nomi di variabili descrittivi e significativi.
- Evitare nomi troppo corti o generici (es. `a, var, x`).

Modulo 2

Variabili e tipi di dati

Tipi di Dati Fondamentali:

- **Numeri:**
  - Interi (`int`): Include numeri sia positivi che negativi senza una componente decimale.
  - Assegnazione diretta: `` `numero = 7, età = 23` ``
  - Derivato da operazioni: `` `somma = 3 + 2, differenza = 10 - 5` ``

  - Conversione da altri tipi di dati: `` `int(3.5) → 3 ` ``
  - Utile per la conversione di stringhe o float in interi quando necessario.

  - Operazioni aritmetiche come addizione (`` `+` ``), sottrazione (`` `-` ``), moltiplicazione (`` `*` ``), divisione (`` `/` ``) , divisione intera (`` `//` ``), modulo (`` `%` ``), potenza (`` `**` ``).
  - Esempio: a = `` `5 * 3 (a diventa 15)` ``

Modulo 2

Variabili e tipi di dati

Tipi di Dati Fondamentali:

- **Numeri:**
  - Floating point (`float`)
  - In Python, i numeri a virgola mobile sono rappresentati dal tipo float.
  - Utilizzati per rappresentare valori numerici con componenti decimali

  - Assegnazione diretta: `` `pi = 3.14, gravità = 9.81` ``
  - Derivato da operazioni: `` `media = (2.5 + 3.7) / 2` ``

  - Conversione da altri tipi di dati: `` ` float(5) → 5.0 ` ``
  - Conversione da e verso stringhe: `` `str(3.14)` ``, `` `float("2.71")` ``

  - Funzioni come `round()` possono essere usate per arrotondare a un certo numero di decimali: `` `round(pi, 2) → 3.14 ` ``
  - Importante considerare gli errori di arrotondamento in calcoli critici.

Modulo 2

Variabili e tipi di dati

Tipi di Dati Fondamentali:

- **Booleani:**
  - In Python, un booleano è un tipo di dato che può essere o `True` (vero) o `False` (falso).
  - Fondamentale per la logica decisionale e il controllo del flusso nei programmi.

  - Assegnazione diretta: `verità = True`, `falso = False`
  - Risultato di espressioni di confronto: `è_maggiore = 5 > 3` (risulta in `True`)

  - Usati per combinare valori booleani.
  - Operatori principali: `and`, `or`, `not`.
  - Esempio: `risultato = (età >= 18) and (età <= 65)`

  - In Python, altri tipi di dati possono essere valutati in un contesto booleano (ad es. in un'istruzione `if`).
  - Tipi "`truthy`": valori non-zero, non-vuoti.
  - Tipi "`falsy`": `0`, `None`, `""` (stringa vuota), `[]` (lista vuota), `{}` (dizionario vuoto).

Modulo 2

Operatori: aritmetici, confronto, logici

Definizione e Utilizzo:

- Utilizzati per eseguire operazioni matematiche comuni.
- Esempi di operatori: `+` (addizione), `-` (sottrazione), `*` (moltiplicazione), `/` (divisione).

Esempi di Operatori Aritmetici:

- **Addizione:** `a + b`
- **Sottrazione:** `a - b`
- **Moltiplicazione:** `a * b`
- **Divisione:** `a / b`
- **Modulo (resto della divisione):** `a % b`
- **Potenza:** `a ** b`
- **Divisione Intera:** `a // b`

Punto Focale:

- Importanza nella manipolazione numerica.
- Utilizzo frequente in calcoli matematici e in logica di programmazione.

Modulo 2

Operatori: aritmetici, confronto, logici

Esempi aritmetici:

```
somma = 0
numero_uno = 5
numero_due = 2
somma = numero_uno + numero_due
print(somma)
7
```

Addizione

```
divisione_con_virgola = 0
numero_uno = 12
numero_due = 5
divisione_con_virgola = numero_uno / numero_due
print(divisione_con_virgola)
2,4
```

Divisione con virgola

```
divisione_senza_virgola = 0
numero_uno = 5
numero_due = 2
divisione_senza_virgola = numero_uno // numero_due
print(divisione_senza_virgola )
2
```

Divisione senza virgola

Modulo 2

Operatori: aritmetici, confronto, logici

Definizione e Utilizzo:

- Usati per confrontare due valori.
- Risultano in un valore booleano: `True` o `False`.

Esempi di Operatori di Confronto:

- **Uguale a:** `a == b`
- **Diverso da:** `a != b`
- **Maggiore di:** `a > b`
- **Minore di:** `a < b`
- **Maggiore o uguale a:** `a >= b`
- **Minore o uguale a:** `a <= b`

Punto Focale:

- Fondamentali nelle decisioni logiche e nei loop.
- Utilizzati in istruzioni condizionali come `if`, `while`.

Modulo 2

Operatori: aritmetici, confronto, logici

Esempi di confronto:

```
numero_uno = 5
numero_due = 5
somma = numero_uno + numero_due
print(somma)
7
```

Uguale a

```
divisione_con_virgola = 0
numero_uno = 12
numero_due = 5
divisione_con_virgola = numero_uno / numero_due
print(divisione_con_virgola)
2,4
```

Diverso da

```
divisione_senza_virgola = 0
numero_uno = 5
numero_due = 2
divisione_senza_virgola = numero_uno // numero_due
print(divisione_senza_virgola )
2
```

Minore o uguale a

Modulo 2

Operatori: aritmetici, confronto, logici

Definizione e Utilizzo:

- Utilizzati per combinare più condizioni logiche.
- Tipicamente restituiscono True o False.

Esempi di Operatori Logici:

- **AND**: `condizione1 and condizione2` (True se entrambe sono vere)
- **OR**: `condizione1 or condizione2` (True se almeno una è vera)
- **NOT**: `not condizione` (Inverte il valore booleano della condizione)

Punto Focale:

- Cruciali nella costruzione di espressioni logiche complesse.
- Spesso usati in controllo di flusso e filtri.

Modulo 2

Operatori: aritmetici, confronto, logici

Esempi di Operatori Logici:

and

A	B	A and B
TRUE	TRUE	TRUE
FALSE	FALSE	FALSE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE

or

A	B	A and B
TRUE	TRUE	TRUE
FALSE	FALSE	FALSE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE

not

A	not A
TRUE	FALSE
FALSE	TRUE

Modulo 2

Stringhe: proprietà e metodi associati

Caratteristiche Fondamentali:

- Le stringhe sono sequenze di caratteri.
- In Python, le stringhe sono immutabili, il che significa che una volta create, il loro contenuto non può essere cambiato.

Dichiarazione di Stringhe

- Le stringhe possono essere create utilizzando singoli (`'stringa'`), doppi (`"stringa"`) o tripli apici (`'''stringa'''` o `"""stringa"""`).
- I tripli apici consentono di creare stringhe su più righe.

Caratteri di Escape

- Caratteri come `\n` (newline) e `\t` (tabulazione) per formattare la stringa.
- Uso del backslash (`\`) per inserire apici all'interno di stringhe: `\'`, `\"`.

Concatenazione e Ripetizione

- Concatenazione: Unione di due stringhe con `+`, es. `'Hello' + 'World'`.
- Ripetizione: Ripetizione di una stringa con `*`, es. `'Hi' * 3`.

Modulo 2

Stringhe: proprietà e metodi associati

Manipolazione di Stringhe

- `.upper()`: converte in maiuscole, es. `'python'.upper()` → `'PYTHON'`.
- `.lower()`: converte in minuscole, es. `'PYTHON'.lower()` → `'python'`.
- `.title()`: ogni parola inizia con una maiuscola, es. `'python is fun'.title()` → `'Python Is Fun' `.

Ricerca e Sostituzione

- `.find(substring)`: restituisce l'indice della prima occorrenza di `substring`.
- `.replace(old, new)`: sostituisce tutte le occorrenze di `old` con `new`.

Stripping

- `.strip()`: rimuove spazi bianchi all'inizio e alla fine.
- `.lstrip()`,`.rstrip()`: versioni specifiche per sinistra e destra.

Modulo 2

Stringhe: proprietà e metodi associati

Suddivisione e Unione

- `.split(separator)`: divide la stringa in una lista basandosi su `separator`.
- `separator.join(list)`: unisce gli elementi di una lista in una stringa, separati da `separator`.

Formattazione

- `.format()`: inserisce i valori all'interno di una stringa, es. `'{} è {}'.format('Python', 'fantastico') `.

Modulo 2

Input ed output: leggere e stampare dati

Ottenere Input dall'Utente

- Uso di `input()` per leggere i dati dall'utente.
- La funzione `input()` cattura l'input come stringa.
- Esempio: `nome = input("Inserisci il tuo nome: ")`

Conversione dei Tipi di Input

- Convertire l'input in altri tipi di dati (es. int, float) per calcoli.
- Esempio: `età = int(input("Inserisci la tua età: "))`

Lettura da File

- Aprire un file con `open()` per leggere i dati.
- Modalità comuni: `'r'` per lettura, `'w'` per scrittura.
- Uso di with `open('file.txt', ‘r’) as file:` per una gestione sicura del file.

Punto Focale

- Importanza della validazione e del controllo degli errori durante la lettura di input.

Modulo 2

Input ed output: leggere e stampare dati

Stampa di Dati

- Uso di `print()` per stampare dati su console.
- Capace di stampare vari tipi di dati, inclusi stringhe, numeri, liste.
- Esempio: `print("Ciao, " + nome)`

Formattazione dell'Output

- Concatenazione di stringhe e variabili.
- Formattazione di stringhe con `.format()` o f-strings (Python 3.6+): `f"Il tuo nome è {nome} e hai {età} anni."`

Scrittura su File

- Uso di `with open('file.txt', 'w') as file:` per scrivere su file.
- Metodo `file.write(testo)` per scrivere sul file.

Punto Focale

- La chiarezza e la leggibilità dell'output sono cruciali per l'interazione utente e il debugging.

Modulo 2

Operazioni condizionali (if, elif, else)

Cos'è un'Operazione Condizionale?

- Un'istruzione che esegue diversi blocchi di codice a seconda che una condizione specifica sia vera o falsa.

Sintassi di Base

- **If:** Esegue il blocco di codice solo se la condizione è vera.
  - `` `if condizione:` ``
  - `` `# Blocco di codice` ``
- **Elif (Else If):** Testa un'altra condizione se la condizione precedente è falsa.
  - `` `elif altra_condizione:` ``
  - `` `# Altro blocco di codice` ``
- **Else:** Esegue un blocco di codice se tutte le condizioni precedenti sono false.
  - `` `else:` ``
  - `` `# Altro blocco di codice` ``

Importanza

- Fondamentale per il controllo del flusso di esecuzione in un programma.
- Permette al programma di reagire a diversi input e situazioni.

Modulo 2

Operazioni condizionali (if, elif, else)

Esempio di if:

```
if temperatura > 30:
    print("Fa caldo oggi!")
```

Esempio di if-elif-else:

```
if temperatura > 30:
    print("Fa caldo oggi!")
elif temperatura < 10:
    print("Fa freddo oggi!")
else:
    print("Che bella giornata!")
```

Modulo 2

Operazioni condizionali (if, elif, else)

Punti da Ricordare

- L'indentazione è cruciale per definire i blocchi di codice.
- Le condizioni vengono valutate in ordine: `if`, poi `elif`, e infine `else`.
- `elif` e `else` sono opzionali e possono essere omessi se non necessari.

Consigli Pratici

- Mantenere le condizioni il più semplice e chiaro possibile.
- Evitare catene eccessivamente lunghe di `elif` per la leggibilità del codice.

Modulo 2

Cicli (for, while) - Introduzione ai Cicli in Python

Cicli in Python: for e while

I cicli sono strumenti fondamentali in programmazione per eseguire un blocco di codice ripetutamente.
Python offre principalmente due tipi di cicli: **for** e **while**.



Ripetere del codice ripetutamente, o "iterazione",
chiamata anche **ciclo** o con il termine inglese **loop**, è
una struttura di controllo, all'interno di
un algoritmo risolutivo di un problema dato, che ordina
all'elaboratore di eseguire ripetutamente
una sequenza di istruzioni, solitamente fino al verificarsi di
particolari.

Modulo 2

Cicli (for, while) - Introduzione ai Cicli in Python

Cicli in Python: for e while

Esistono due forme di ciclo principalmente (da cui emergono poi le altre eventuali) :

quelle di schema **MENTRE-FINE-MENTRE** e **RIPETI-FINCHÉ condizione**.



MENTRE (condizione)
blocco di istruzioni
FINE-MENTRE

e

RIPETI
blocco di istruzioni
FINCHÉ condizione

Ogni linguaggio implementa poi nel concreto i cicli praticamente fornendo strumenti per realizzare cicli del primo o del secondo tipo o entrambi. In python non esistono strumenti specifici equivalenti ad entrambi di tipi di ciclo, ma possono essere ottenuti ugualmente entrambi come vedremo.

Modulo 2

Cicli (for, while) - Introduzione ai Cicli in Python

Il Ciclo "for" in Python

Il ciclo for è una struttura di controllo in Python che consente di scorrere tutti gli elementi di una sequenza (come una lista, una tupla, una stringa o un range) ed eseguire un blocco di codice in modo ripetuto per ogni elemento di tale sequenza.

Il ciclo for è comunemente utilizzato quando il numero di iterazioni è predefinito o quando si vuole eseguire una serie di istruzioni per ogni elemento di una collezione di dati.

```
for item in sequence:
    ...
    ...
    ...
```

Modulo 2

Cicli (for, while) - Introduzione ai Cicli in Python

Il Ciclo "for" in Python

Sintassi: for item in sequence:
Dove sequence è, come suggerisce il nome, una sequenza, come liste, tuple, stringhe.
Con range() è possibile generare una sequenza

Es: **range(5)**
Genera una sequenza di tipo range (che è una struttura particolare) che contiene i numeri [0, 1, 2, 3, 4] (tuttavia non è una lista o un array, non va confuso).

Dunque, possiamo scrivere:

```
for i in range(5):
    print(i)
```

Ottenendo

```
PS C:\projects\python\testvari> python test.py
0
1
2
3
4
```

Modulo 2

Cicli (for, while) - Introduzione ai Cicli in Python

Il Ciclo "while" in Python

I cicli **while** sono la forma più generica di ciclo.
La struttura tipica è:

```
while condition:
        ...
        ...
        ...
```

il blocco di codice indentato finché la condizione della **while** è vera.

Esempio:

```
count = 0
while count < 5:
    print(count)
    count += 1
```

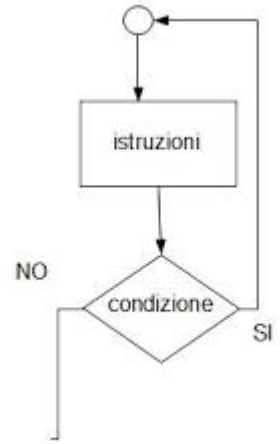
Il che produrrà

```
PS C:\projects\python\testvari> python test.py
0
1
2
3
4
```

Modulo 2

Cicli (for, while) - Introduzione ai Cicli in Python

Corrispondenza tra Python ed i MENTRE-FINE-MENTRE ed il RIPETI-FINCHÉ condizione



Ora, se ricordiamo, abbiamo detto che ci sono due tipologie di cicli:
il **MENTRE-FINE-MENTRE** ed il **RIPETI-FINCHÉ condizione**.
Il primo

MENTRE (condizione)
blocco di istruzioni
FINE-MENTRE

È perfettamente rappresentato da while e for
Nel **while** la struttura è evidente e nel for la condizione corrisponde all'aver terminato lo scorrere della sequenza.
Mentre il

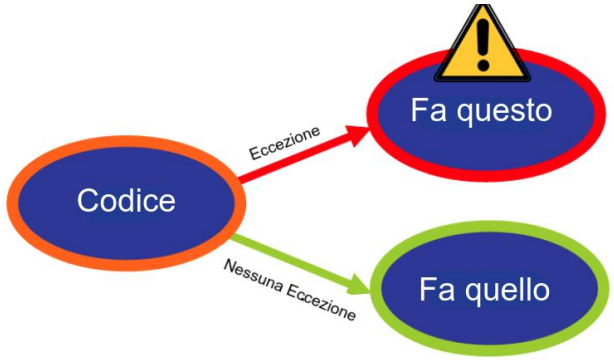
RIPETI
blocco di istruzioni
FINCHÉ condizione

Non ha strutture direttamente così implementate. Tuttavia, il comportamento può essere simulato utilizzando

```
while True:
    # blocco di istruzioni
    if condizione:
        break
```

Modulo 2

Gestione delle eccezioni (try, except, else e finally)



La gestione degli errori

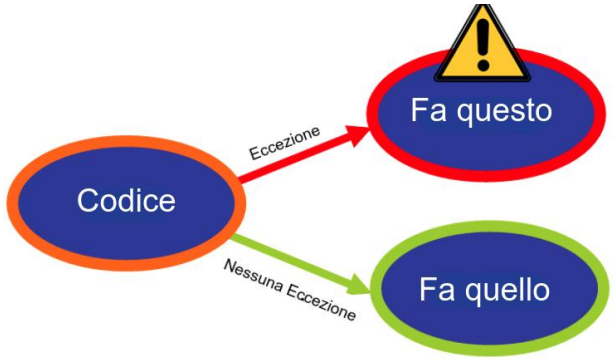
La gestione delle eccezioni in Python è un meccanismo cruciale per il controllo del flusso del programma. Esso consente di gestire e rispondere ad errori che possono verificarsi durante l'esecuzione del codice. Le parole chiave principali per la gestione delle eccezioni sono **try**, **except**, **else** e **finally**.

Modulo 2

Gestione delle eccezioni (try, except, else e finally)

try, except, else e finally

Ecco come funzionano:



try: Questo blocco viene utilizzato per avvolgere il codice che può potenzialmente causare un'eccezione. Python esegue il codice all'interno del blocco try come normale, ma se si verifica un errore, invece di arrestare il programma, Python cerca un blocco except corrispondente e lo esegue.

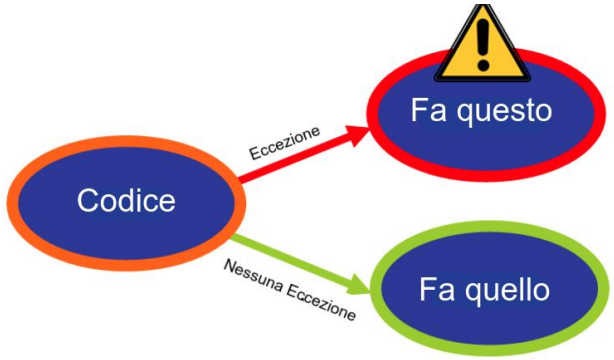
except: Dopo un blocco try, segue uno o più blocchi except, che catturano e gestiscono le eccezioni. Un blocco except può specificare un tipo di eccezione particolare da catturare o può essere scritto senza specificare nulla, per catturare qualsiasi eccezione. È possibile avere più blocchi except per gestire tipi diversi di eccezioni in modi diversi.

else: (Opzionale) Questo blocco viene eseguito se non si verificano eccezioni nel blocco try. È utile per codice che deve essere eseguito solo se il blocco try non ha generato un'eccezione.

finally: (Opzionale) Questo blocco viene eseguito indipendentemente dal fatto che si sia verificata un'eccezione o meno. È utile per azioni di pulizia che devono essere eseguite indipendentemente dall'esito del codice nel blocco try. Ad esempio, può essere utilizzato per chiudere file o rilasciare risorse esterne.

Modulo 2

Gestione delle eccezioni (try, except, else e finally)



try, except, else e finally

Ecco un semplice esempio di come queste parole chiave possono essere utilizzate in un blocco di gestione delle eccezioni:

```
try:
    # Codice che potrebbe generare un'eccezione
    risultato = 10 / 0
except ZeroDivisionError:
    # Gestisce specificamente l'eccezione ZeroDivisionError
    print("Non è possibile dividere per zero.")
except Exception as e:
    # Gestisce qualsiasi eccezione non catturata dai blocchi
except precedenti
    print(f"Si è verificato un errore: {e}")
else:
    # Viene eseguito solo se non si verificano eccezioni nel
blocco try
    print("La divisione è stata eseguita con successo.")
finally:
    # Viene sempre eseguito, indipendentemente dall'esito del
blocco try
    print("Questo viene eseguito sempre, con o senza eccezione.")
```

In questo esempio, se 10 / 0 viene eseguito, si verificherà un'eccezione ZeroDivisionError e verrà gestito dal corrispondente blocco except. Il blocco finally, che stampa un messaggio, verrà eseguito indipendentemente da ciò che succede nei blocchi try e except.

Modulo 2

Funzioni: definizione, parametri, return

Funzioni (definizione)

Una funzione è definita utilizzando la parola chiave def, seguita da un nome di funzione, parentesi e un blocco di codice. Ecco un esempio di come si definisce una funzione:

```
def saluta(nome):
    print(f"Ciao, {nome}!")
```

In questo esempio, saluta è il nome della funzione, e nome è un **parametro** che la funzione riceve.

Modulo 2

Funzioni: definizione, parametri, return

Nota del termine e sua origine

Il termine **funzione** ha collegamenti con il termine **applicazione** ed entrambi i termini sono correlati sia al significato algoritmico che di restituzione valore **tipicamente matematici**, infatti anche in matematica le funzioni si dicono anche applicazioni e possono restituire valori, ma al contempo sono rappresentabili anche algoritmicamente

Matematica
In matematica, una **funzione** (o applicazione) è una relazione tra un insieme di input e un insieme di output possibili con la proprietà che ogni input è relazionato a esattamente un output. Una funzione matematica prende un argomento (x), lo processa e restituisce un risultato (y), comunemente descritto come f(x) = y.

Le funzioni matematiche possono essere rappresentate in vari modi, come:

Espressioni algebriche (ad es. f(x) = x^2)
Grafici
Tabelle di valori
Descrizioni verbali
Queste rappresentazioni possono essere viste come **algoritmi** nel senso che definiscono una serie di istruzioni per arrivare da un input a un output.

Modulo 2

Funzioni: definizione, parametri, return

Nota del termine e sua origine

Analogamente in informatica

Informatica
In informatica, una **funzione** (o a volte chiamata **subroutine**, **procedura** o **metodo** in diversi linguaggi di programmazione o contesti) è un **costrutto** che permette di raggruppare **un insieme di istruzioni** per eseguire un compito specifico (**algoritmo**). Come le funzioni matematiche, le funzioni in programmazione prendono degli input (parametri), eseguono delle operazioni (il corpo della funzione), e spesso restituiscono un valore (tramite l'istruzione return).

Anche se in matematica le funzioni sono tipicamente pure e non hanno effetti collaterali, le funzioni in programmazione possono avere effetti collaterali (come modificare lo stato di un oggetto o scrivere su un file) oltre a calcolare e restituire valori.

Modulo 2

Funzioni: definizione, parametri, return

Nota del termine e sua origine

Collegamenti
Il concetto di funzione come mappatura da un insieme di input a un insieme di output è fondamentale sia in matematica che in informatica. In entrambi i casi, l'enfasi è sulla relazione tra gli input e gli output, e sul processo di trasformazione dell'input in output. La nozione di funzione come processo ripetibile ed isolato è centrale per entrambe le discipline, e in entrambe le aree, l'approccio algoritmico alla descrizione e all'analisi delle funzioni è di vitale importanza.

Modulo 2

Funzioni: definizione, parametri, return

Parametri

I parametri sono variabili che agiscono come segnaposto per i valori che vengono passati alla funzione al momento della sua chiamata. Una funzione può avere più parametri o nessuno. I parametri vengono specificati tra le parentesi nella definizione della funzione.

Esistono diversi tipi di parametri in Python:

Parametri Posizionali: Devono essere passati nell'ordine in cui sono definiti.
Parametri Parola Chiave (Keyword): Possono essere passati in qualsiasi ordine specificando il nome del parametro.
Parametri Predefiniti: Hanno un valore di default che viene usato se non viene fornito alcun valore al momento della chiamata.
Args e Kwargs: Permettono di passare un numero variabile di parametri posizionali (*args) o di parola chiave (**kwargs).

Più avanti approfondiremo questi diversi tipi di parametri

Modulo 2

Funzioni: definizione, parametri, return

Return (resituzione di valori)

Una funzione può restituire un valore al codice che la chiama utilizzando la parola chiave **return**. Questo termina l'esecuzione della funzione e "restituisce" il risultato. Se **return** non è presente o non c'è un valore dopo return, la funzione restituisce **None**, che è il default (e concettualmente indica che non è stato restituito nessun valore, anche se in realtà **None** è, comunque, un valore).

```
def somma(a, b):
    return a + b

risultato = somma(5, 3)  # risultato è 8
```

Modulo 2

Funzioni: definizione, parametri, return

Return (resituzione di valori)

Le funzioni possono anche restituire più valori sotto forma di tuple

```
def operazioni(a, b):
    return a+b, a-b

somma, differenza = operazioni(10, 5)  # somma è 15 e differenza è 5
```

In sintesi, le funzioni sono uno strumento essenziale per scrivere codice ordinato, organizzato e riutilizzabile in Python. Le loro capacità di accettare parametri e restituire valori rendono il codice molto più flessibile e potente.

Modulo 2

I diversi tipi di parametri

Vediamo, ora le varie tipologie di paramtri, come ci eravamo ripromessi.
Ricordiamo le tipologie qui:

Riepilogo dei diversi tipi di parametri in Python:

Parametri Posizionali: Devono essere passati nell'ordine in cui sono definiti.
Parametri Parola Chiave (Keyword): Possono essere passati in qualsiasi ordine specificando il nome del parametro.
Parametri Predefiniti: Hanno un valore di default che viene usato se non viene fornito alcun valore al momento della chiamata.
Args e Kwargs: Permettono di passare un numero variabile di parametri posizionali (*args) o di parola chiave (**kwargs).

Modulo 2

I diversi tipi di parametri

Parametri posizionali

Questi sono i parametri più comuni. Devono essere passati alla funzione esattamente nell'ordine in cui sono definiti. Ad esempio:

```
def descrivi_animali(primo, secondo):
    print(f"Il primo animale è un {primo}, il secondo è un
{secondo}.")

descrizione_animali('cane', 'gatto')
```

In questo esempio, **'cane**' sarà assegnato a **primo** e **'gatto**' a **secondo**.

Modulo 2

I diversi tipi di parametri

Parametri Parola Chiave (Keyword)

Quando si chiama una funzione, è possibile passare gli argomenti come coppie chiave-valore. Questo permette di non dover rispettare l'ordine dei parametri, perché vengono identificati per nome:

```
def descrivi_animali(primo, secondo):
    print(f"Il primo animale è un {primo}, il secondo è un
{secondo}.")

descrizione_animali(secondo='gatto', primo='cane')
```

Il risultato sarà lo stesso dell'esempio precedente perché i nomi dei parametri vengono usati per passare gli argomenti.

Modulo 2

I diversi tipi di parametri

Parametri Predefiniti

Un parametro predefinito ha un valore di default che viene usato se nessun argomento viene passato per quel parametro:

```
def descrivi_animali(primo, secondo='criceto'):
    print(f"Il primo animale è un {primo}, il secondo è un {secondo}.")

descrizione_animali(primo='cane')
```

Qui, se secondo non viene specificato, verrà utilizzato il valore predefinito 'criceto'.

Modulo 2

I diversi tipi di parametri

Args e Kwargs

*args e **kwargs consentono di passare un numero variabile di argomenti a una funzione. *args è usato per passare una lista di parametri posizionali, mentre **kwargs permette di passare un dizionario di parametri parola chiave.:

```
def funzione_con_variabili(*args, **kwargs):
    for arg in args:
        print(arg)
    for key in kwargs:
        print(key, kwargs[key])

funzione_con_variabili('cane', 'gatto', primo='criceto',
secondo='coniglio')
```

Nell'esempio, *args cattura 'cane' e 'gatto', mentre **kwargs cattura {'primo': 'criceto', 'secondo': 'coniglio'}.

Questi meccanismi rendono le funzioni Python estremamente flessibili, consentendo agli sviluppatori di costruire interfacce di funzioni che possono gestire un'ampia varietà di casi d'uso con una sola definizione di funzione.

Modulo 2

I diversi tipi di parametri

ATTENZIONE PERO'

Se provi a chiamare la funzione funzione_con_variabili con il seguente codice:

```
funzione_con_variabili('cane', 'gatto', secondo='criceto', 'coniglio')
```

Dove ricordiamo che la funzione era così definita

```
def funzione_con_variabili(*args, **kwargs):
    for arg in args:
        print(arg)
    for key in kwargs:
        print(key, kwargs[key])
```

Otteniamo un errore: **kwargs deve avere specificata una **chiave**

```
def funzione_con_variabili(*args, **kwargs):
    for arg in args:
        print(arg)
    for key in kwargs:
        print(key, kwargs[key])

funzione_con_variabili('cane', 'gatto', secondo='criceto', 'coniglio')
```

Peraltro, vscode darebbe anche un segnale di **errore**

Modulo 2

Scope delle variabili e namespaces

I concetti in generale

In Python, lo "scope" di una variabile determina la parte del programma in cui si può accedere alla variabile e a quale "namespace" appartiene. Un "namespace" è un insieme di nomi di variabili (identificatori) e le informazioni sulle variabili che quei nomi rappresentano.

Modulo 2

Scope delle variabili e namespaces

Namespace
(concetto)

Un **namespace** è una mappatura da nomi (come i nomi delle variabili) a oggetti (come i dati che quelle variabili rappresentano). In Python, ogni ambito ha il suo namespace. Questo aiuta a evitare conflitti di nomi poiché lo stesso nome può essere utilizzato in namespace diversi senza ambiguità. I namespace sono implementati in Python come dei dizionari interni.

Modulo 2

Scope delle variabili e namespaces

Scope delle variabili
(concetto)

Lo **scope di una variabile** si riferisce alla regione del codice in cui una variabile è visibile.

Python segue la regola **LEGB**, che definisce l'ordine in cui gli scope sono cercati per risolvere un nome:

Local (Locale): Il namespace all'interno di una funzione o blocco di codice locale. Se una variabile viene definita all'interno di una funzione, il suo scope è locale a quella funzione.
Enclosing (Includente): Gli scope degli ambiti circostanti, da quello più interno a quello più esterno. Questi sono gli scope relativi a tutte le funzioni "enclosing" o "non locali" in cui si trova attualmente il codice.
Global (Globale): Il namespace a livello di modulo o file. Le variabili definite al livello più alto di un file Python sono globali e visibili in tutto il codice nel file stesso.
Built-in (Incorporato): Il namespace che contiene le funzioni built-in e le variabili di Python. Questo è lo scope più esterno.

Modulo 2

Scope delle variabili e namespaces

Python usa queste regole per determinare da quale namespace prendere il valore di una variabile quando viene incontrata. Se una variabile viene utilizzata all'interno di una funzione senza essere definita all'interno di essa, Python cercherà nei namespace circostanti nell'ordine **LEGB** finché non trova il nome variabile. Se il nome non viene trovato, Python solleverà un NameError.

```
x = 10  # Variabile globale

def outer_function():
    y = 5  # Variabile enclosing

    def inner_function():
        z = 3  # Variabile locale
        print(x, y, z)  # Accede a tutte le variabili

    inner_function()

outer_function()
```

In questo esempio, **x** è una variabile globale, **y** è una variabile nel scope enclosing (perché è definita in **outer_function** che è la funzione che include **inner_function**), e **z** è una variabile locale a **inner_function**.

Modulo 2

Modificare Variabili Globali

Per modificare una variabile globale all'interno di una funzione, è necessario dichiararla con la parola chiave global:

```
x = 10

def change_global():
    global x
    x = 20

change_global()
print(x)  # Stampa 20
```

NB: La comprensione degli scope e dei namespace è fondamentale per evitare conflitti di nomi e per scrivere codice Python chiaro e manutenibile.

Modulo 2

Modificare Variabili Non Locali

Per modificare una variabile in uno scope enclosing, si può usare la parola chiave nonlocal:

```
def outer_function():
    y = 5

    def inner_function():
        nonlocal y
        y = 10

    inner_function()
    print(y)  # Stampa 10

outer_function()
```

NB: La comprensione degli scope e dei namespace è fondamentale per evitare conflitti di nomi e per scrivere codice Python chiaro e manutenibile.

Modulo 2

Moduli e pacchetti: organizzazione del codice

Moduli

Un modulo è un file Python con estensione .py che contiene definizioni e istruzioni Python. Il nome del modulo è lo stesso del nome del file. Un modulo può definire funzioni, classi e variabili che possono essere riutilizzate importandole in altri moduli o script.

Modulo 2

Moduli e pacchetti: organizzazione del codice

Facciamo un esempio Moduli

Supponiamo di avere da fonte esterna o realizzato da noi un file chiamato mio_modulo.py con il seguente contenuto:

```
# mio_modulo.py

def saluta(nome):
    print(f"Ciao, {nome}!")
```

In tal caso potrai utilizzare le funzioni definite in mio_modulo.py in un altro file Python nello stesso modo:

```
# altro_file.py

import mio_modulo

mio_modulo.saluta("Alice")
```

Modulo 2

Moduli e pacchetti: organizzazione del codice

Importare una funzione specifica

Puoi anche importare specifiche definizioni da un modulo usando la forma from module import something:

```
from mio_modulo import saluta

saluta("Bob")
```

Modulo 2

Moduli e pacchetti: organizzazione del codice

Pacchetti

Un pacchetto è una directory di moduli Python che contiene un file speciale chiamato __init__.py. Il file __init__.py può essere vuoto, ma la sua presenza indica che la directory è un pacchetto Python, e può quindi essere importata in altri moduli e script. I pacchetti sono un modo per strutturare i moduli Python in namespace, consentendo così di utilizzare nomi di moduli identici senza conflitti.
Per esempio, se hai una struttura di directory come questa:

```
mio_pacchetto/
    __init__.py
    sub_pacchetto/
        __init__.py
        mio_modulo.py
```

Puoi importare mio_modulo che si trova dentro sub_pacchetto che a sua volta si trova dentro mio_pacchetto:

```
from mio_pacchetto.sub_pacchetto import mio_modulo

mio_modulo.saluta("Carla")
```

Modulo 2

Moduli e pacchetti: organizzazione del codice

Sub-pacchetti

I pacchetti possono includere anche sub-pacchetti per gestire meglio la complessità e organizzare il codice in moduli logici, facilitando la comprensione e il riutilizzo del codice.

Python stesso viene fornito con una vasta libreria standard, che è un insieme di moduli e pacchetti che forniscono funzionalità comuni, come l'interazione con il file system, l'accesso a rete, e molto altro.

Modulo 2

Moduli e pacchetti: organizzazione del codice

Installazione di Pacchetti Esterni

Al di là dei moduli e pacchetti che crei tu o quelli che vengono con la libreria standard di Python, puoi installare pacchetti di terze parti usando strumenti come pip, il gestore di pacchetti Python. Questi pacchetti sono spesso ospitati su PyPI (Python Package Index) e possono essere installati con un semplice comando:

```
pip install nome_del_pacchetto
```

In conclusione, moduli e pacchetti sono essenziali per mantenere il codice ben organizzato, chiaro e manutenibile, specialmente in progetti più grandi e complessi. Consentono di dividere il codice in componenti logici che possono essere sviluppati, testati e mantenuti in modo più efficiente.

Modulo 2

Introduzione ai commenti e alla documentazione

In Python, i commenti e la documentazione sono strumenti fondamentali per rendere il codice più leggibile e manutenibile. Entrambi forniscono spiegazioni e chiarimenti senza influenzare l'esecuzione del programma.

Commenti

Al di là dei moduli e pacchetti che crei tu o quelli che vengono con la libreria standard di Python, puoi installare pacchetti di terze parti usando strumenti come pip, il gestore di pacchetti Python. Questi pacchetti sono spesso ospitati su PyPI (Python Package Index) e possono essere installati con un semplice comando:

Modulo 2

Introduzione ai commenti e alla documentazione

Commenti in singola linea

I commenti su una singola linea iniziano con il carattere #. Tutto ciò che segue il # sulla stessa riga è un commento.

```
# Questo è un commento su una singola linea
print("Ciao, mondo!")  # Questo è un commento dopo una istruzione
```

Modulo 2

Introduzione ai commenti e alla documentazione

Commenti multilinea (più linee singole e combinazioni di tasti)

Python non ha un meccanismo dedicato per commenti multilinia simili a /* ... */ in altri linguaggi.
Di norma si dovrebbe procedere con più commenti in singola linea.
Con editor come VSCode esistono, talvolta, delle combinazioni di tasti per commentare e decommentare rapidamente blocchi di righe selezionate; in VS code:
CRTL+K+C per commentare
CRTL+K+U per decommentare

```
# Questo è un commento su più linee.
# Viene utilizzato per commenti più lunghi.

print("Ciao, mondo!")
```

Modulo 2

Introduzione ai commenti e alla documentazione

Commenti multilinea (stringhe)

Inoltre, convenzionalmente, le stringhe non assegnate vengono spesso utilizzate come commenti multilinia, anche se il loro scopo primario non è quello di essere dei commenti.

```
"""
Questo è un commento su più linee.
Viene utilizzato per commenti più lunghi.
"""
print("Ciao, mondo!")
```

Modulo 2

Introduzione ai commenti e alla documentazione

Documentazione (Docstrings)

I docstrings sono stringhe letterali che appaiono come la prima istruzione in una definizione di modulo, funzione, classe o metodo. A differenza dei commenti, i docstrings sono oggetti stringa effettivi e possono essere accessibili a runtime usando l'attributo .__doc__ della funzione, classe, ecc.

Funzioni e Metodi

I docstrings sono circondati da tripli doppi apici (""") e spesso contengono una spiegazione dell'oggetto, i suoi parametri, il valore di ritorno e informazioni sulle eccezioni sollevate.

```
def somma(a, b):
    """
    Restituisce la somma di due numeri.

    Parametri:
    a (int): Il primo addendo.
    b (int): Il secondo addendo.

    Restituisce:
    int: La somma di a e b.
    """
    return a + b
```

Modulo 2

Introduzione ai commenti e alla documentazione

Documentazione (Docstrings)

Moduli e classi

I moduli ed altre strutture che vedremo, le classi, dovrebbero anche avere docstrings che spiegano cosa fanno e come dovrebbero essere utilizzati.

```
"""
Questo modulo fornisce funzioni matematiche di base.

Il modulo `matematica` include funzioni per eseguire operazioni aritmetiche
come somma, sottrazione, moltiplicazione e divisione.

Esempio di utilizzo:
    import matematica
    risultato = matematica.somma(10, 5)
    print(risultato)

Funzioni:
- somma(a, b): Restituisce la somma di a e b.
- sottrai(a, b): Restituisce la differenza tra a e b.
- moltiplica(a, b): Restituisce il prodotto di a e b.
- dividi(a, b): Restituisce il quoziente di a e b.
"""

def somma(a, b):
    """
    Restituisce la somma di due numeri.

    Parametri:
    a (int/float): Il primo addendo.
    b (int/float): Il secondo addendo.

    Restituisce:
    int/float: La somma di a e b.
    """
    return a + b

# Altre funzioni (sottrai, moltiplica, dividi) seguono qui...
```

Modulo 2

Introduzione ai commenti e alla documentazione

Documentazione (Docstrings)

Moduli e classi

Nell'esempio che segue c'è una definzione di classe, come la vedremo più avanti e la documentazione relativa

```
class Persona:
    """
    Classe che rappresenta una persona.

    Attributi:
    nome (str): Il nome della persona.
    età (int): L'età della persona.
    """

    def __init__(self, nome, età):
        self.nome = nome
        self.età = età
```